

Help Menu

The commands are described under the [Help Menu](#).

Methods of the WorkerPool

_Methods of the WorkerPool

The *WorkerPool* provides:

- The *methods* listed in the table of contents to the left.
- The [Methods of All Objects](#).

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window [Show Attributes and Methods](#). The figure below illustrates the information using the example of the object *Station*.

Name	Value	Inherited	Watchable	Signature
contentsList	[]			([Contents:table]) -> any
createAttr				(NameOfUserDefinedAttribute:string, DataType:string) -> boolean
createIcon				(IconName:string, Width:integer, Height:integer) -> integer
deleteAttr				(NameOfUserDefinedAttribute:string) -> boolean

Name of the method Identifier and data type of the parameter you enter Data type of the return value

Name	Value	Inherited	Watchable	Signature
moveToFolder				(Folder:object) -> object
MU	VOID			([Index:integer:=1]) -> object
MUPart	VOID			([Index:integer:=1]) -> object

Name of the method Identifier and data type of the parameter you enter Default value Data type of the return value

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected [Class \[general description\]](#).
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected [Instance \[general description\]](#).

An example of the **Syntax** line of the individual methods might look like this:

```
<Path>.openDialog([CallOpenControl:boolean:=false]) → boolean
```

- The expression *<Path>* designates the path of the object to which the *method* applies.
- The signature of the method, consisting of the identifier and the data type of the parameter, is listed in parentheses. The expression *(Parameter:string)*, for example, designates a parameter of data type *string*. Instead of a constant value, you can also use a variable of the required type or a *method* that returns the required data type.

Note

Make sure to enter the parentheses for expressions within parentheses (...). Not entering them may lead to unexpected results and open the *Debugger*.

Optional parameters are listed within brackets. The expression *[,Parameter:boolean]*, for example, means that you can, but do not have to enter the *boolean* parameter.

If a parameter has a default value, the signature shows the default value after the parameter, *:= false* in the example above.

If the *method* has a return value, the signature shows its data type after the arrow *->*, *→ boolean* in the example above.

getAssignedWorker [SimTalk]

Returns the assigned *Worker* designated by the number in the *WorkerPool* designated by <Path>.

Type

Method

Syntax

```
<Path>.getAssignedWorker(No:integer) -> object
```

Parameter

The parameter *No* of data type *integer* designates the number of the *Worker* that is assigned.

Return Value

The return value has the data type *object*.

Example

```
print MyWorkerPool.getAssignedWorker(2)  
-- might, for example, return .Resources.John:2
```

See also

[Assigned Workers \[WorkerPool\]](#)

getAssignedWorkersTable [SimTalk]

Returns the table that contains the assigned *Workers* of the *WorkerPool* designated by <Path>.

Syntax

```
<Path>.getAssignedWorkersTable([AssignedWorkers:table]) -> any
```

Type

Method

Parameter

The optional parameter *AssignedWorkers* of data type *table* designates the name of the table into which the *Workers* will be written.

If you do not specify the optional parameter, *Plant Simulation* returns an array with the assigned *Workers*.

Return Value

The return value has the data type *any*.

Example

```
MyWorkerPool.getAssignedWorkersTable(MyAssignedWorkersTable)  
MyWorkerPool.getAssignedWorkersTable
```

See also

[Assigned Workers \[WorkerPool\]](#)

getIdleWorker [SimTalk]

Returns the first idle *Worker* in the *WorkerPool* designated by <Path>.

Remarks

getIdleWorker returns the first idle *Worker* for whom the attribute *IsIdle* is to true and sets it to false. When you no longer need the *Worker*, you can send the *Worker* to the *WorkerPool* and set *IsIdle* to true.

Type

Method

Syntax

```
<Path>.getIdleWorker -> object
```

Return Value

The return value has the data type *object*.

Is the first idle *Worker* in the *WorkerPool*.

VOID if no *Worker* is idle.

Example

```
.Resources.WorkerPool.getIdleWorker
```

SimTalk

[IsIdle \[SimTalk\] - Worker](#)

[NumIdleWorkers \[SimTalk\]](#)

getWorkersToCreateTable [SimTalk]

Returns the **Workers to Create** table of the *WorkerPool* designated by <Path> and writes it into a table.

Remarks

Plant Simulation always resolves **relative paths** in relation to the *Frame* in which the *WorkerPool* is inserted. If *Plant Simulation* cannot resolve a path in the **Scope** array during the simulation, it shows an error message and asks if you would like to stop the simulation.

This also applies to the **Home Location**.

Type

Method

Syntax

```
<Path>.getWorkersToCreateTable(WorkersToCreateTable:table/void[, byref  
Inherited:boolean])
```

Parameters

You can specify the following parameters:

- The parameter *WorkersToCreateTable* of data type *table* designates the name of the table that contains the *Workers* to be created. The column of the table has to have the data type *table*.
If the optional parameter *Inherited* is passed, you can specify VOID for the parameter *WorkersToCreateTable*. In this case *WorkersToCreateTable* is ignored.
- The optional parameter *Inherited* designates a local variable of data type *boolean* which is set to true if the *WorkersToCreateTable* of the *WorkerPool* is inherited. It is set to false if it is not inherited.

Example

```
var rt: table                                // read the Workers to Create  
table  
MyWorkerPool.getWorkersToCreateTable(rt)    // set the new amount  
rt [2,1]:= AmountOfWorkers                 // name of a Variable  
  
// assign the Workers to Create table to the WorkerPool  
MyWorkerPool.setWorkersToCreateTable(rt)
```

SimTalk

[setWorkersToCreateTable \[SimTalk\]](#)

getWorkersToCreateTableRow [SimTalk] - WorkerPool

See also

[Workers to Create](#)

[Home Location](#)

[Tab Scope](#)

[The Relative Path](#)

getWorkersToCreateTableRow [SimTalk] - WorkerPool

The sub-attributes of the method `getWorkersToCreateTableRow` set and get the individual settings of the **Workers to Create table** of the *WorkerPool* designated by `<Path>`.

Remarks

Plant Simulation always resolves **relative paths** in relation to the *Frame* in which the *WorkerPool* is inserted. If *Plant Simulation* cannot resolve a path in the **Scope** array during the simulation, it shows an error message and asks if you would like to stop the simulation.

This also applies to the **Home Location**.

Type

Method

Syntax

```
<Path>.getWorkersToCreateTableRow(row:integer).AdditionalServices:string[]
<Path>.getWorkersToCreateTableRow(row:integer).Amount:integer
<Path>.getWorkersToCreateTableRow(row:integer).Efficiency:real
<Path>.getWorkersToCreateTableRow(row:integer).HomeLocation:path
<Path>.getWorkersToCreateTableRow(row:integer).Scope:array
<Path>.getWorkersToCreateTableRow(row:integer).Shift:string
<Path>.getWorkersToCreateTableRow(row:integer).Speed:speed
<Path>.getWorkersToCreateTableRow(row:integer).Worker:object
```

Parameters

You can specify the following parameters:

- The parameter *row* of data type *integer* designates the row in the **Workers to Create** table in which you would like to set the attribute.

	Worker	Amount	Shift	Speed	Efficiency	Home Location	Scope	Additional Services
1	*.Resources.Worker	1				Home1	Station1; Station2	-
2	*.Resources.Worker	1				Home11	Station11; Station21	

- The parameter *AdditionalServices* is an *array* of data type *string* containing the additional services that the *Worker* can provide.

- The parameter *Amount* of data type *integer* designates the number of *Workers* that are going to be created.
- The parameter *Efficiency* of data type *real* designates the efficiency of the *Worker*.
- The parameter *HomeLocation* of data type *path* designates the path to the *Workplace* to which the *Worker* walks once he has finished his current job.
- The parameter *Scope* of data type *array* designates the path to the material flow objects to which the *Worker* can be brokered. The scope also encompasses *sub-Frames*.
- The parameter *Shift* of data type *string* designates the shift during which the *Worker* works. You define the shift itself in the assigned [ShiftCalendar](#).
- The parameter *Speed* of data type *speed* designates the speed with which the *Worker* walks.
- The parameter *Worker* of data type *object* sets the class of the *Worker* which the *WorkerPool* uses as the template for the *Workers* to be created.

Example

```
MyWorkerPool.getWorkersToCreateTableRow(1).AdditionalServices := ["drill1",
"drill2", "drill3"]
```

SimTalk

getWorkersToCreateTableRow.[AdditionalServices](#) [SimTalk]

getWorkersToCreateTableRow.[Amount](#) [SimTalk] - WorkerPool

getWorkersToCreateTableRow.[Efficiency](#) [SimTalk] - WorkerPool

getWorkersToCreateTableRow.[HomeLocation](#) [SimTalk] - WorkerPool

getWorkersToCreateTableRow.[Scope](#) [SimTalk] - WorkerPool

getWorkersToCreateTableRow.[Shift](#) [SimTalk] - WorkerPool

getWorkersToCreateTableRow.[Speed](#) [SimTalk] - WorkerPool

getWorkersToCreateTableRow.[Worker](#) [SimTalk] - WorkerPool

[getWorkersToCreateTable](#) [SimTalk]

[setWorkersToCreateTable](#) [SimTalk]

See also

[Workers to Create](#)

Tab Scope

Home Location

The Relative Path

getWorkingWorkersTable [SimTalk]

Returns the table that contains the working *Workers* of the *WorkerPool* designated by <Path>.

Type

Method

Syntax

```
<Path>.getWorkingWorkersTable(WorkingWorkers:table)
```

Parameter

The parameter *WorkingWorkers* of data type *table* designates the name of the table into which the working *Workers* will be written.

Example

```
MyWorkerPool.getWorkingWorkersTable(MyWorkingWorkers)
```

See also

[Working Workers \[WorkerPool\]](#)

setWorkersToCreateTable [SimTalk]

Sets the name of the **Workers to Create** table for the *Workers* of the *WorkerPool* designated by <Path>.

Remarks

Plant Simulation always resolves **relative paths** in relation to the *Frame* in which the *WorkerPool* is inserted. If *Plant Simulation* cannot resolve a path in the **Scope** array during the simulation, it shows an error message and asks if you would like to stop the simulation.

This also applies to the **Home Location**.

Type

Method

Syntax

```
<Path>.setWorkersToCreateTable(WorkersToCreateTable:table/void)
```

Parameter

Specify **void** for the parameter to activate inheritance of the **Workers to Create** table in the *WorkerPool* instead of filling the table with the *Workers* that you entered into the table.

Note

To apply the changes to the *Workers to Create* table to the simulation run, the change has to occur before the model is initialized.

The parameter *WorkersToCreateTable* of data type *table* designates the **Workers to Create table**. Enter the **Names** of the *Workers* to be created, the name of the **Shift**, their **Amount**, their **Speed**, their **Efficiency**, their **Home Location**, their **Scope**, and **Additional Services** into the table.

Note

The name is not case-sensitive, just like the names of *attributes* and *methods* of the objects are not case-sensitive.

To save memory and improve access speed, all places which are using such a case-insensitive string are pointing to the same string in main memory. The visible and unexpected result is that the first occurrence of the string defines how the string is written in terms of upper- and lower-casing.

In *SimTalk* you can compare strings in an case-insensitive manner with the `~=` operator, compare [Relational Operators](#).

Examples

```
MyWorkerPool.setWorkersToCreateTable(MyWorkersToCreateTable)
```

```
var rt: table // read the Workers to Create
table
MyWorkerPool.getWorkersToCreateTableRow(rt) // set the new amount
rt [2,1]:= AmountOfWorkers // name of a Variable
// assign the Workers to
Create table
// to the WorkerPool
MyWorkerPool.setWorkersToCreateTable(rt)
```

SimTalk

[getWorkersToCreateTable \[SimTalk\]](#)

[getWorkersToCreateTableRow \[SimTalk\] - WorkerPool](#)

See also

[Workers to Create](#)

[Tab Scope](#)

[Home Location](#)

[The Relative Path](#)

[Relational Operators](#)

Read-Only Attributes of the WorkerPool

Read-Only Attributes of the WorkerPool

The *WorkerPool* provides these read-only attributes:

- The *read-only attributes* listed in the table of contents to the left.
- The [_Read-Only Attributes of All Objects](#).

You can query the values of the *read-only attributes*, but you cannot set them as *Plant Simulation* computes the value for the point-in-time at which you query it. In most cases a *read-only attribute* corresponds to an unavailable dialog item on one of the tabs of the object, for example on the tab **Statistics**.